Master Team Project Summer 2024
ClassCompanion
Milestone 2

Team 1

Yagubkhan Yagubkhanov, Team Lead / Github Master
yagubkhan.yagubkhanov@informatik.hs-fulda.de

Samman Adhikari, Frontend Lead

Shantanu Patil, Backend Lead

Aralikere Yogananda Gunarachana

Mariha Asif

Amber Asad

Date: 04.06.2024

Revision Summary:

Date	Revision	Summary
27.05.2024	1.0	Initial draft created
04.06.2024	1.1	Made changes based on instructor's feedback, group chat feature added to USP as the priority 1

Content and structure for Milestone 2

1. Functional Requirements - Prioritized
2. List of main data items and entities
3. UI Mockups and Storyboards
4. High level Architecture, Database Organization
5. High Level UML Diagrams
6. Key Risks for the Project Server Related Issues
7. Project management

1. Functional Requirements - Prioritized

Priority 1 Requirements (Must Have)

Registered Users (Students and Tutors)

1. **User Registration and Login**

   1.1 Students and tutors must register using their university email address.

   1.2 Registration process should include validation of email addresses to ensure eligibility.

   1.3 Registered users (students and tutors) should be able to log in and log out securely.

   1.4 Check box to register as student or tutor.

2. **Profile Management**

   2.1 Users (students and tutors) should be able to manage their profiles, including updating personal information, adding skills, and uploading profile pictures.

   2.2 Students should not be able to search for other students based on skills or subjects in tutor search.

3. **Tutor Search Functionality**

   3.1 Students should be able to search for tutors based on skills, subjects, or keywords.

   3.2 Search results should display a list of relevant tutors based on the search criteria.

4. **Live Chat Feature**

   4.1 Students should be able to initiate live chat sessions with tutors they want to contact.

   4.2 Chat functionality should support text messages, file uploads (such as documents or images), and other multimedia content.

   4.3 If a tutor is offline, messages should be delivered to their inbox for later viewing.

5. **Inbox Management**

   5.1 Both tutors and students should have access to their respective inboxes.

5.2 Inboxes should display messages received from other users, including chat messages and notifications.

5.3 Users should be able to mark messages as read, reply to messages, and delete messages as needed.

6. **Admin Panel**

   6.1 An admin panel should be available for administrators to manage user accounts, monitor chat activity, and handle any reported issues.

   6.2 Admins should have the ability to view user profiles, moderate content, and enforce site policies.

   6.3 Admins can moderate (hide or delete) inappropriate content. (USP)

7. **Group Chat Feature**

   7.1 Group chat functionality so a tutor can connect with multiple students interested in the same topic simultaneously.

Priority 2 Requirements (Desired)

Guest Users

1. **Guest Access**
   - Guest students and teachers should be allowed to access the website, but they cannot utilize chat functionality until they register with their university email address.

Priority 3 Requirements (Opportunistic)

Registered Users (Students and Tutors)

1. **Calendar Integration**
   - Integration of a calendar feature where tutors and students can schedule sessions and view availability.

Admin Panel

2. **Advanced Admin Features**
   - Additional tools for administrators to generate reports on usage statistics and user engagement.
   - Admins should be able to create announcements or broadcast messages to all users.

Summary

By organizing and prioritizing these requirements, we ensure that the most critical features necessary for the core functionality and user experience are developed first. This approach allows us to allocate resources effectively and iteratively improve the product by integrating additional features as time and resources permit.

2. List of main data items and entities

1. Type of Users:

Student

- ID: Unique identifier for each student.
- Name: Full name of the student.
- Username: Chosen username for login.
- Email: Email address provided by HS Fulda.
- Password: Encrypted password for account security.
- Contact Number: Phone number for communication.
- Address: Physical address.
- Date of Birth: For age verification.

Tutor

- ID: Unique identifier for each tutor.
- Name: Full name of the tutor.
- Username: Chosen username for login.
- Email: Email address provided by HS Fulda.
- Password: Encrypted password for account security.
- Contact Number: Phone number for communication.
- Address: Physical address.
- Academic Qualification: Highest level of education.
- Experience: Teaching experience and subject expertise.
- Rates: Hourly rates.
- Ratings: Average rating from student feedback.

Administrator

- ID: Unique identifier for each administrator.
- Name: Full name of the administrator.
- Username: Chosen username for login.
- Email: Email address provided by HS Fulda.
- Password: Encrypted password for account security.
- Contact Number: Phone number for communication.

- Roles: Specific administrative roles and access levels.

2. Student/Tutor Profile

- Personal Information: Name, gender, date of birth, and contact details.
- Contact Details: Phone number, email address, and physical address.
- Professional Experience: Highest level of Education, teaching experience and subject expertise (for tutors).
- Profile Picture: A photograph for profile identification.
- CV/Resume: Detailed professional document (for tutors).

3. Messages

- Message ID: Unique identifier for each message.
- Sender ID: User ID of the message sender.
- Receiver ID: User ID of the message receiver.
- Timestamp: Date and time when the message was sent.
- Content: Text content of the message.
- Status: Read or unread status.

4. Feedback

- Feedback ID: Unique identifier for each feedback entry.
- Student ID: User ID of the student providing feedback.
- Tutor ID: User ID of the tutor being reviewed.
- Rating: Numeric rating (e.g.1-5 stars).
- Timestamp: Date and time when the rating was submitted.

5. Administration

- User Management: Tools for managing students, tutor accounts.
- Content Moderation: Tools for moderating shared messages.

6. Search Functionality

- Search Query: Keywords entered by the user.
- Filters: Options to filter results by tutor rating and experience.
- Search Results: List of tutors or courses matching the search criteria.
- Sorting Options: Options to sort results by experience, rating, etc.

7. Multimedia

- Multimedia ID: Unique identifier for each multimedia item.
- Uploader ID: User ID of the tutor uploading the content.
- File Type: Type of file (e.g., PDF, video, audio).

- File Size: Size of the file.
- Upload Date: Date and time when the file was uploaded.

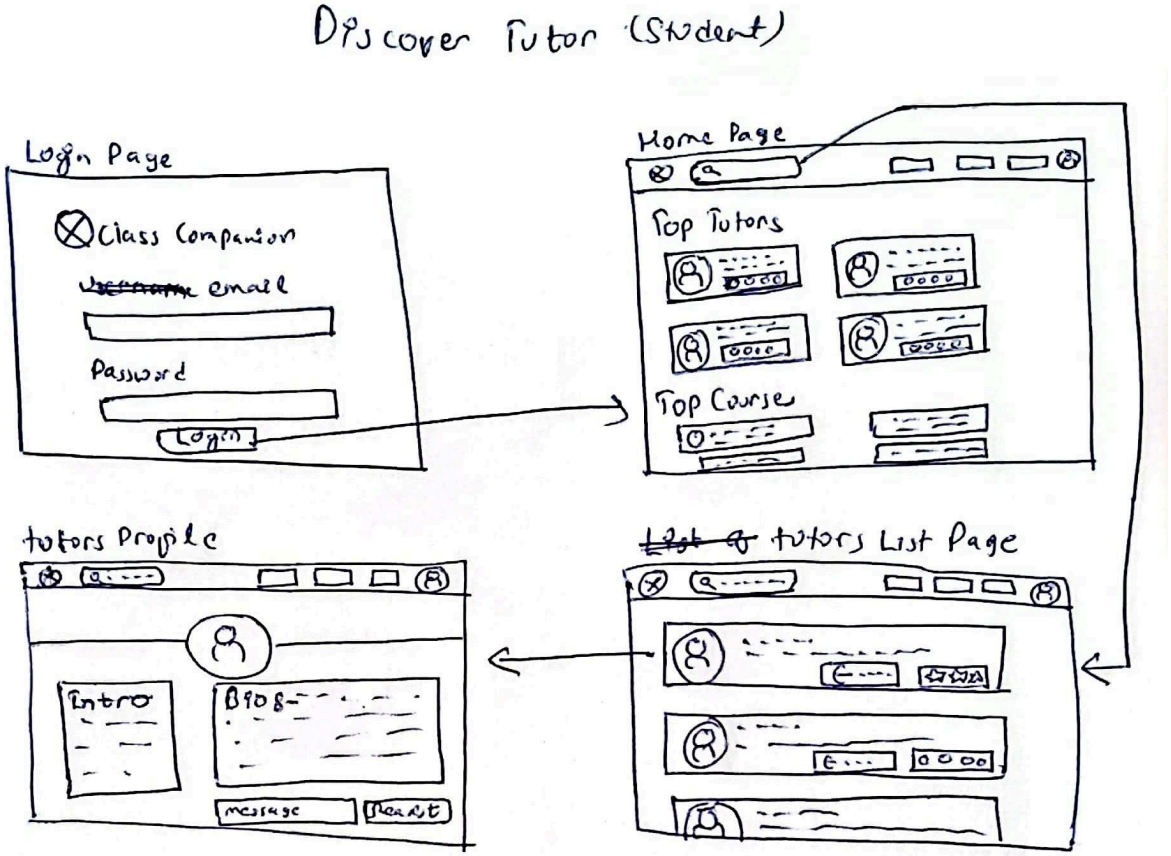
8. Groups

- Group ID: Unique identifier for each group entry.
- Name: Group name

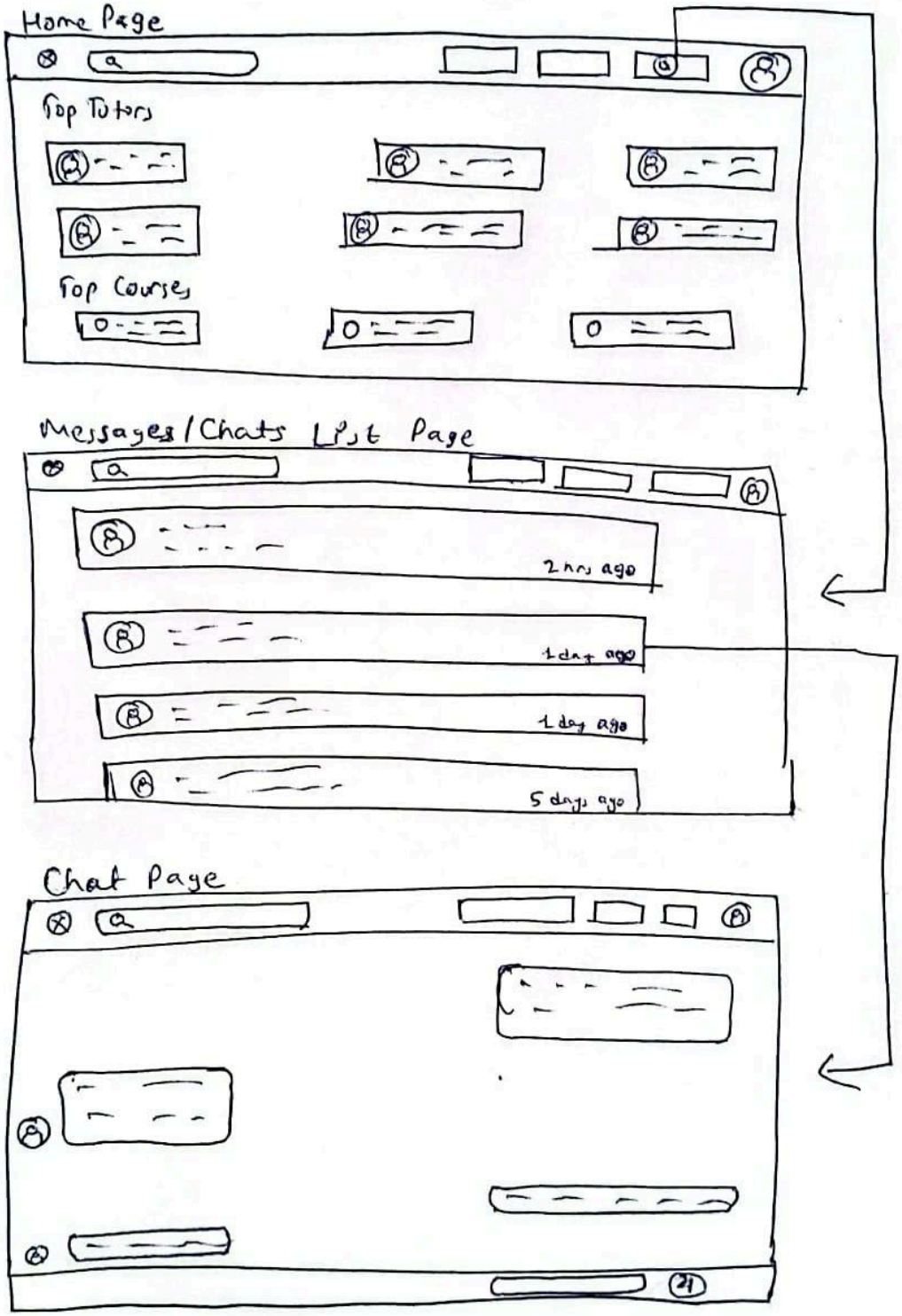
9. Message Group

- Message Group ID: Unique identifier for each message group entry
- Group ID: Identifier of the associated group
- Message ID: Identifier of the associated message

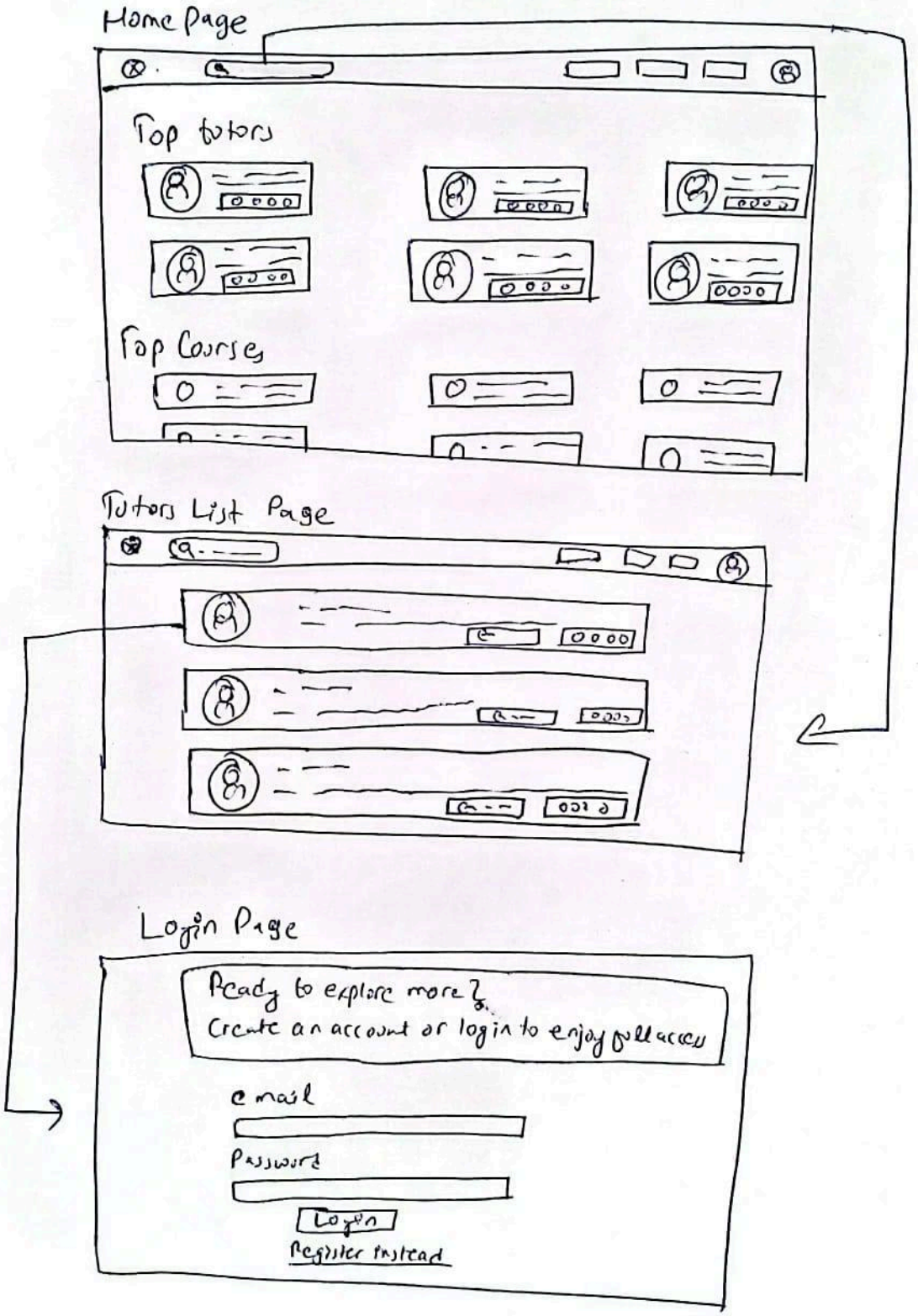
3. UI Mockups and Storyboards

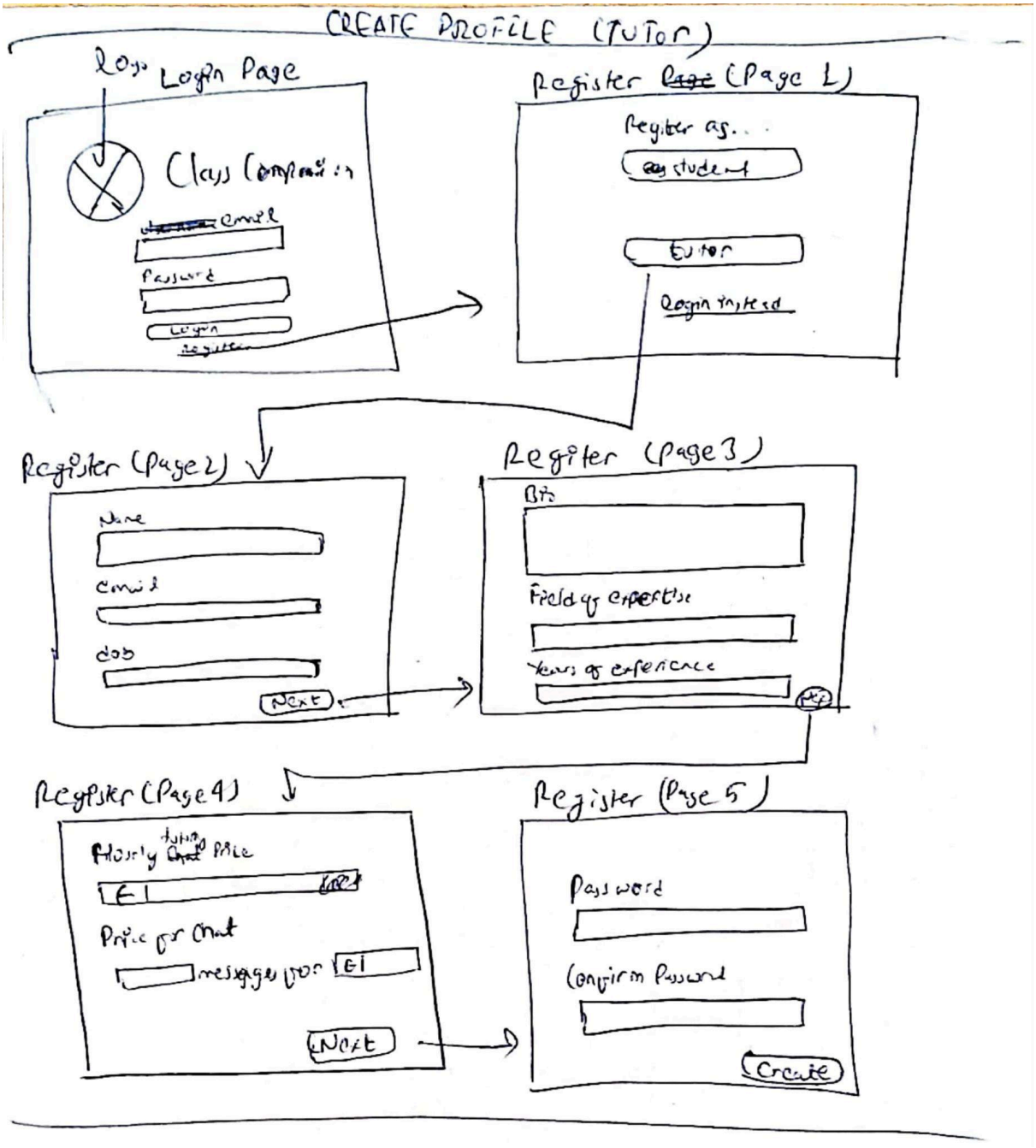


Ask tutors help (student)



Guest Access (Guest)

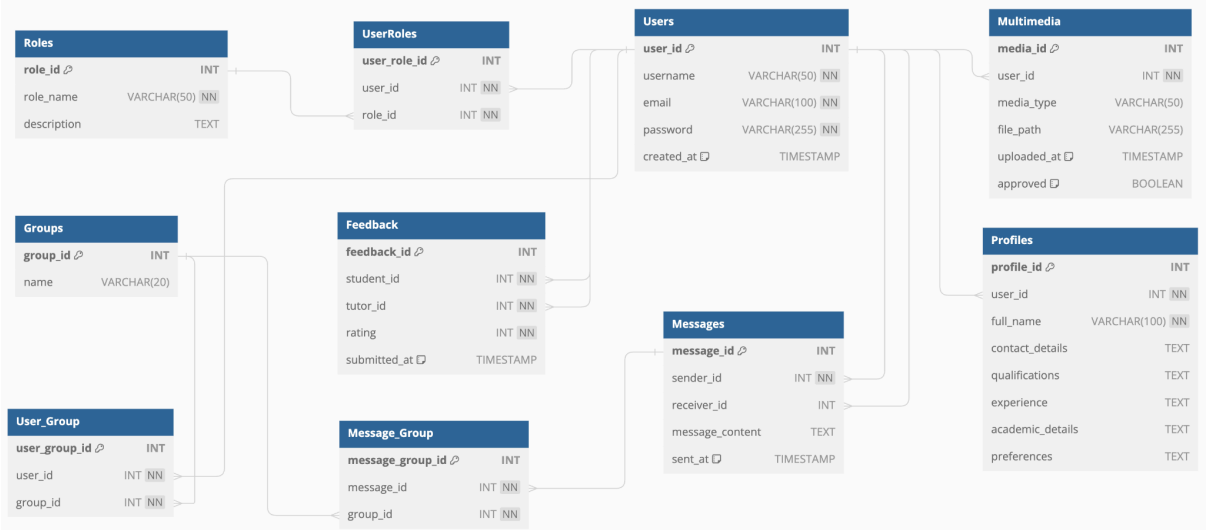



CREATE PROFILE (TUTOR)
Login Page
Class
email
Password
Login
Register
Register (Page 1)
Register as...
as student
Tutor
Register (Page 2)
Name
email
dob
Next
Register (Page 3)
Bio
Field of expertise
Years of experience
Register (Page 4)
Hourly tutoring Price
Price per chat
messages per
Next
Register (Page 5)
Password
Confirm Password
Create

4. High level Architecture, Database Organization

The database schema includes several main tables to manage users, roles, permissions, profiles, messages, services, feedback, administration, search, and multimedia. Below is a detailed schema that includes RBAC and the additional features.

1. **Users Table**
Stores information about the users in the system.
2. **Roles Table**
Stores the different roles that users can have.
3. **UserRoles Table**
Associates users with roles. A user can have multiple roles.
4. **Profiles Table**
Stores comprehensive information about all registered students and tutors.
5. **Messages Table**
Stores messages exchanged between users.
6. **Feedback Table**
Stores student ratings of tutors and reviews about their tutoring experiences.
7. **Multimedia Table**
Stores information about multimedia files uploaded by users.
8. **Groups Table**
Stores information about groups.
9. **User_Group Table**
Associates users with groups. A user can have multiple groups.
10. **Message_Group Table**
Associates messages with groups.



Media Storage

Multimedia files (e.g., CVs, files, lectures) will be stored in a dedicated file system AWS S3. The database will store metadata and file paths to ensure quick access and management.

Search/Filter Architecture and Implementation

The search functionality will be implemented using SQL with the %LIKE% operator for text searches. Indexed columns will ensure faster search performance.

Search Algorithm Steps:

1. User inputs keywords into the search bar.
2. The system constructs a SQL query using the %LIKE% operator to match keywords with indexed columns.
3. The query is executed, and the results are fetched from the database.
4. Results are formatted and displayed to the user.

APIs

A set of RESTful APIs will be created to handle various functionalities:

1. User Management API
    - Register, login, update profile, delete account
2. Messaging API
    - Send, receive, delete messages
3. Feedback API
    - Submit, view, delete feedback
4. Search API
    - Perform searches based on keywords

Significant Algorithms and Processes

Rating and Ranking Algorithm

Tutors will be ranked based on average ratings, number of reviews, and recency of feedback. A weighted average formula will be used to prioritize recent and highly-rated tutors. To rank tutors effectively, we can use the formula that considers the following factors:

Average Rating: The average rating given by students.
Review Count is the total number of reviews received by the tutor.
Recency Score is calculated based on the timestamps of the reviews, giving more weight to recent reviews.

Here's a proposed weighted average formula:

Tutor Score = ($w1$ × Average Rating) + ($w2$ × Review Count) + ($w3$ × Recency Score)

Where: $w1$, $w2$, $w3$ are the weights assigned to each factor, respectively.

Example Calculation

Assume a tutor has the following metrics:

Average Rating: 4.6
Review Count: 50
Number of Reviews in the Last 30 Days: 10

Let's assume the following example weights:

$w1$ = 0.5
$w2$ = 0.3
$w3$ = 0.2

Combining the factors, we get:

Tutor Score = (0.5 × Average Rating) + (0.3 × Review Count) + (0.2 × Recency Score) = (0.5 × 4.6) + (0.3 × 50) + (0.2 × 10) = 19.3

SQL Implementation

To implement this simplified formula in SQL, we can use a single query to calculate and rank tutors:

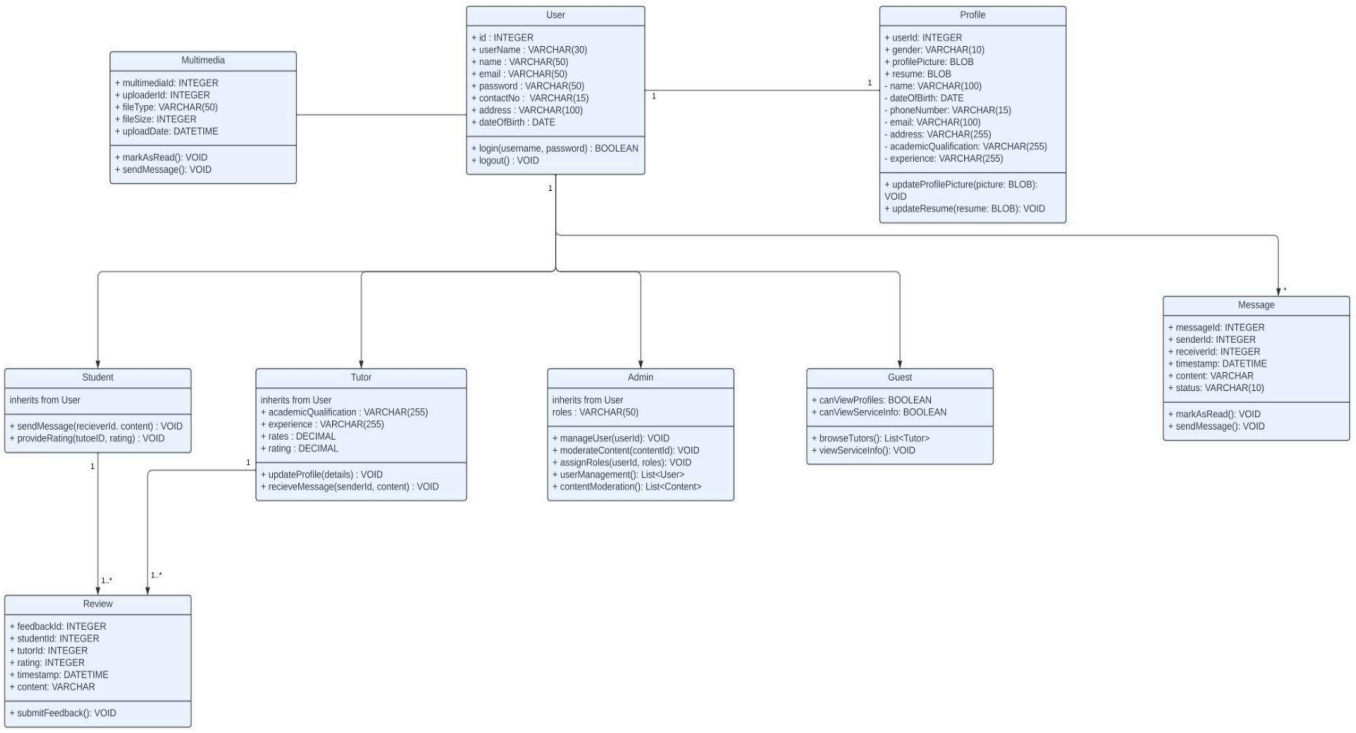
```
SELECT
  tutor_id,
  (0.5 * AVG(rating)) +
  (0.3 * COUNT(feedback_id)) +
  (0.2 * SUM(CASE
    WHEN DATEDIFF(CURRENT_DATE, submitted_at) <= 30 THEN 1
    ELSE 0
  END)) AS tutor_score
FROM
  Feedback
GROUP BY
  tutor_id
ORDER BY
  tutor_score DESC;
```

Software Tools and Frameworks

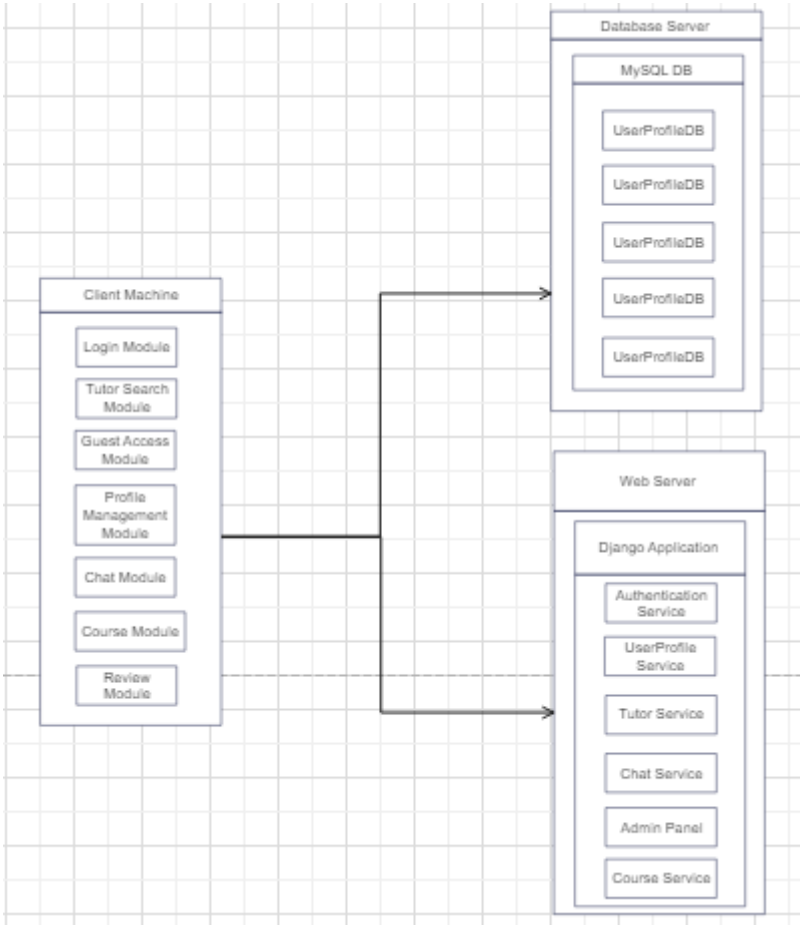
Media Storage: AWS S3 for secure and scalable file storage.

5. High Level UML Diagrams

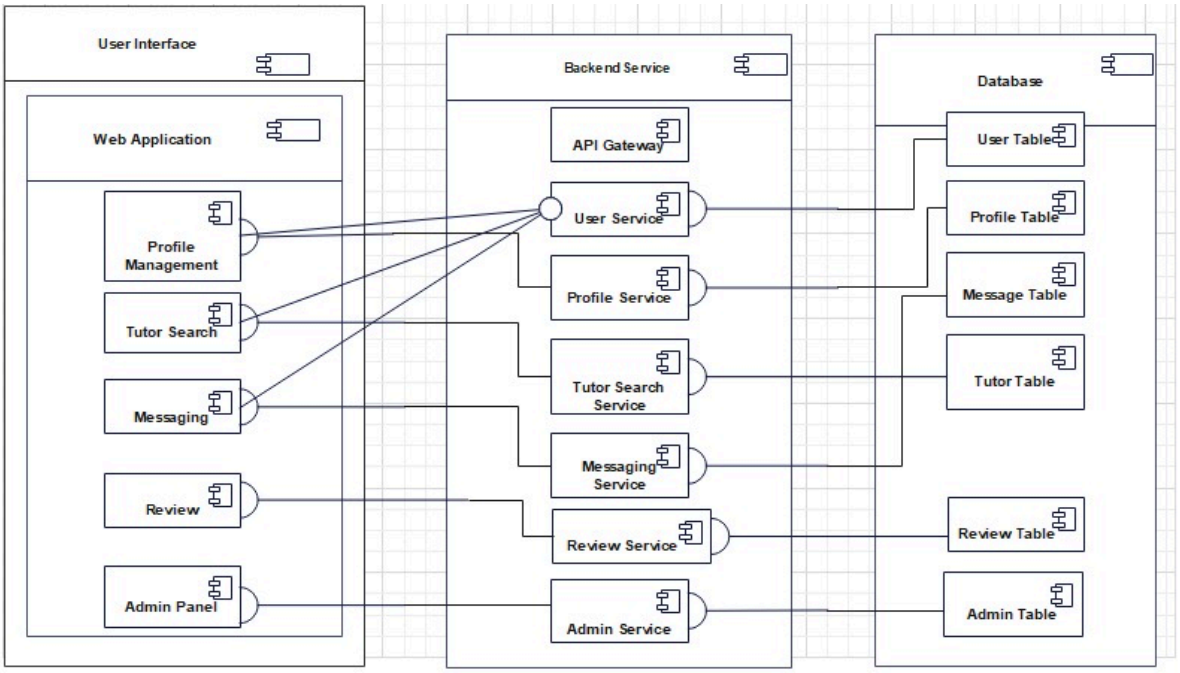
UML class diagrams



Deployment diagram



Component diagram



6. Key Risks for the Project Server Related Issues

Server Related Issues:

Risk: Issues related to AWS or other hosting services, such as unreliable systems or AWS credit issues, could disrupt the project's progress.
Resolution: Maintain a standby AWS account in an unused state to be activated immediately if the main account's credit expires. Additionally, pre-plan alternative hosting options to switch quickly if one fails, ensuring continuous service availability.

Git Related Issues:

Risk: Accidental faulty commits or deletion of significant code portions by team members can lead to application breakdowns and data loss.
Resolution: Create local backups periodically to prevent substantial data loss. Always pay close attention to PRs and merge only if approved. In case of data loss, recover code from backups.

Skill-Related Issues:

Risk: Team members may encounter difficulties in delivering solutions promptly, particularly if they are still learning new technologies or processes.
Resolution: Redistributing tasks under the guidance of front-end or back-end leads helps balance the workload. Providing additional training and resources supports team members in enhancing their skills, ensuring timely project completion.

Team Collaboration Issues:

Risk: Lack of simultaneous collaboration on features requiring both front-end and back-end input can cause bottlenecks and delays.
Resolution: Ensure proper communication through regular meetings and collaborative tools. Re-plan task priorities to keep everyone in sync and avoid delays. Foster a culture of teamwork and shared responsibility.

Software/Hardware Related Issues:

Risk: Breakdown of personal laptops or PCs, or software issues like code editors, compilers, interpreters, or operating system corruption can halt individual contributions and slow down the team's progress.
Resolution: Factor in possible equipment failures when preparing the timeline to ensure deadlines are not affected.

Technical Dependency Issues:

Risk: Compatibility issues with frameworks or tools (e.g., a Python or JS library incompatible with certain browsers or OS) can require significant rework.
Resolution: Regularly test for compatibility issues throughout the development process. Allocate extra time in the timeline for potential changes in libraries or source code adjustments. Have a backup plan for alternative technologies if needed.

Team Member's Health or Personal Issues:

Risk: Significant absenteeism from team members due to health or personal issues can affect the project timeline and delivery.
Resolution: Encourage the team to give extra time to the project if necessary or renegotiate deadlines to accommodate unforeseen circumstances. Maintain a flexible task allocation system to cover for absent members and ensure continuous progress.

Legal Content Dependency Issues:

Risk: Delays in obtaining required permissions from third parties or the university can halt progress on specific features that depend on external data.
Resolution: Continue development using dummy data that closely resembles the required data. Integrate the actual data once permissions are obtained without significant rework. Maintain clear communication with third parties to expedite the approval process.

7. Project management

Strategy

To ensure that the front-end and back-end teams can operate independently while maintaining effective collaboration, we will implement a structured project management strategy. This strategy includes clear task assignment, setting deadlines, and using appropriate tools for tracking progress. The focus is on creating well-defined interfaces and endpoints that both teams agree upon, facilitating seamless integration.

Tools and Methods

1. Task Management Tool: GitHub

- GitHub will be our primary tool for managing tasks. We will use the “issues” feature of Github to track tasks.
- Labels and Milestones: Issues will be organized using labels (e.g., "frontend," "backend," "bug," "enhancement") and milestones to categorize and prioritize tasks.
- Assignees and Deadlines: Each issue will have an assigned person responsible for its completion and a deadline to ensure accountability and timely progress.

2. Regular Meetings

- Daily Chat on a messenger: Instead of daily stand-ups, team members will check in via a group chat on a messaging platform (Whatsapp). This will include updates on what they worked on the previous day, what they plan to work on today, and any blockers they are facing. This approach ensures continuous communication and quick resolution of issues.

- Weekly Syncs: A weekly meeting for both teams to discuss progress, review common interfaces, and ensure alignment on goals and deadlines.

3. Clear Interface Agreements

- API Documentation: We will use tools like Swagger to create comprehensive API documentation that both teams can reference. This ensures that the front-end team knows exactly what data to expect from the back-end and how to interact with it.
- Mock Servers: For the front-end team to work independently, we will set up mock servers that simulate the API responses. This allows the front-end team to develop and test their features without depending on the back-end development being completed.